CaseBridge — Team Plan and Phased Build (SIH26264)

Integrated Dashboard for Monitoring and Tracking Institution-Related Complaint Cases Across AICTE Bureaus/Cells Team of 6. 19 phases (0-18). Each phase has an owner, deliverables and acceptance checks.

1. One-line pitch

One institution → one consolidated case view → clear ownership and history → **explainable, human-reviewed overlap alerts** → coordinated action → measurable reports.

Our differentiator: every overlap alert shows its evidence (same Permanent ID, shared reference, similar topic, close dates). Nothing is ever auto-merged or auto-closed. We report measured precision and recall on labelled synthetic case pairs.

2. Team and ownership

Member	Role	Owns (phases)
Adarsh	Lead, architecture, hard backend	0, 2, 11, 12, 17 (+ code review on all)
Rohitha	Research, core backend	1, 5
Sohana	Flexible full-stack, integration	6, 14, 15, 16
Satya Kamal	UI/UX + frontend lead	8, 9, 13
Priya	PPT, demo, presenter, frontend (case screens)	3, 10, 18
Shreeshanth	Data engineer, QA	4, 7, data-quality checks in 16, test checklists in all phases

Support pairings: Rohitha with Adarsh on the schema; Shreeshanth with Sohana on search and exports; Priya with Satya on components; Priya with Rohitha on the PPT references.

2b. Compressed schedule (deadlines: project finishing by Tue 27 Oct, PPT submitted Wed 28 Oct)

Nineteen phases cannot all be built at full depth in four days, so phases run in parallel tracks and some are cut to a minimum. Feature freeze: **Mon 26 Oct, 9 pm**. After that, only bug fixes.

Day	Adarsh	Rohitha	Sohana	Shreeshanth	Satya Kamal	Priya
Fri 23	Repo, schema, API contract + mock JSON (0, 2)	Assumptions doc (1); CRUD scaffold (5)	Backend scaffold with Rohitha; events/transfer endpoints (6)	Scenario spec from Adarsh; synthetic data generator + seeded DB (4)	Wireframes + design system (8)	PPT outline; slides 1-2 draft (3)
Sat 24	Overlap stage 1, start stage 2 (11, 12)	Case and institution CRUD, profile endpoint (5)	Workflow, alert logic (6, 14)	Labelled pairs file; search/filters (7)	App shell + dashboard (9); start overlap queue layout (13)	Slides 3-6 draft (morning); case list + case detail (10)
Sun 25	Finish similarity + eval numbers (12); API fixes	References for slide 6; fix data and CRUD issues	Reports (15); mock roles (16)	Data-quality checks; tests; success checklist on finished parts	Overlap queue UI with the live matching API (13); dashboard fixes	Institution profile (10); alerts and reports UI components; PPT v2 with architecture diagram
Mon 26	Integration; full flow works end to end	Fix data issues; help with bugs	Integration, regression start	Bug triage and retests	Polish; empty/error states	PPT v3 with real screenshots and eval numbers; demo script draft**. Freeze 9 pm**

Day	Adarsh	Rohitha	Sohana	Shreeshanth	Satya Kamal	Priya
Tue 27	Regression x3 on a clean clone; final fixes	Bug fixes; docs	Deploy	Final checklist	Demo video recording/editing	PPT final review (Adarsh + one other); rehearse demo. **Project in finishing status by 6 pm** **Submit PPT before noon.** No code changes.
Wed 28	Support	Support	Support	Support	Support	

If we fall behind, cut in this order: (1) sentence-transformer embeddings (keep TF-IDF + rapidfuzz), (2) CSV/PDF export, (3) admin screen and real auth (keep mock roles), (4) region-wise report (keep Bureau, status, ageing), (5) PostgreSQL (stay on SQLite). **Never cut:** institution profile, case history, overlap queue with reasons, dashboard counts, the precision/recall result.

3. Working rules (from the Setu build)

1. **One phase at a time.** Write a short plan, get a lead "ok", then code.
2. **Verify, don't trust.** "It works" needs evidence: a screenshot, a DB query, test output. Never accept a summary alone.
3. **Commit after each verified phase.** One branch per person, pull request reviewed by Adarsh, merge to `main`.
4. **Do not display raw/unresolved data.** The UI always reads resolved fields, for example the institution's name from the institution table, not a copy typed into the case.
5. **Counts come from one source.** Dashboard, case list and reports all query the same case records.
6. **Synthetic data only.** Label it so on slides and in the UI footer.
7. **No unproven claims.** No "official SLA", "zero duplicates" or "100% accuracy". Report only measured numbers.
8. **Daily 15-minute sync.** Done yesterday, doing today, blocked on.

4. Tech stack and why

Layer	Choice	Why
Frontend	React (Vite) + React Router + TypeScript + Tailwind	An internal, logged-in dashboard needs no SEO or server rendering, so a single-page app is simpler. The team knows React; Vite starts fast. API URL comes from `VITE_API_URL`.
Backend	Python + FastAPI + Pydantic	Pydantic validates every request and response; auto API docs; the matching code is Python
Database	SQLite for dev, PostgreSQL if time	Zero setup for the prototype; same SQL
ORM	SQLModel / SQLAlchemy	Typed models, easy migrations
Matching	rule engine + rapidfuzz + TF-IDF (scikit-learn); optional sentence-transformers	Explainable first, semantic as a supporting signal
Charts	Recharts	Simple dashboard charts
Auth	Mock JWT with four roles	Enough to show role-based access
Tests	pytest + a manual checklist	Backend logic tested, UI verified by hand
Deploy	Vercel (frontend), Render (backend)	Free tiers

5. Data model (draft; finalise in Phase 2)

- **institutions:** id, permanent_id (unique), name, address, region, state, type
- **cases:** id, case_no (unique), institution_id, bureau, category, description, received_date, status, assigned_officer, last_action, last_action_date, next_action, expected_disposal_date, priority
- **case_events:** id, case_id, ts, actor, bureau, event_type (created, status_change, transfer, escalation, remark, link), from_bureau, to_bureau, remarks, attachment_ref. *Append-only.*
- **overlap_links:** id, case_a_id, case_b_id, reasons (JSON), confidence_band, review_status (open, related, duplicate, not_related, needs_review), reviewed_by, reviewed_at, note
- **alerts:** id, type (overlap, pending, escalation, sla), case_id, link_id, created_at, due_at, status

- **users:** id, name, role, bureau
- **config:** key, value (for example `sla_warn_days = 3`, labelled as demo)

  Statuses: pending, under_process, disposed, transferred, escalated. **Bureaus (demo):** RB, PGRC, plus labelled demo cells such as "Regional Office (demo)". Confirm real values with the problem owner and label ours as assumptions.

6. Matching design (the core)

1. **Resolve the institution:** exact Permanent ID. If missing or conflicting, flag for manual verification. Never guess.
2. **Candidate pairs:** open cases for the same institution in different Bureaus/Cells.
3. **Signals, each recorded as a reason:**
   - same Permanent ID
   - same or related category
   - shared reference (letter number, case number mentioned in text)
   - text similarity (TF-IDF cosine, rapidfuzz token-set ratio; optional sentence embeddings)
   - date proximity
4. **Output:** a confidence band (High / Medium / Low) from transparent rules, plus the list of reasons and both case IDs. No opaque score.
5. **Human decision:** related / duplicate / not related / needs more review, plus a coordination note. Cases keep separate IDs and histories.
6. **Evaluation:** a labelled synthetic set of at least 200 pairs with noise. Report precision and recall against the baseline "same institution ID only".

7. Phases

Phase 0: Kickoff and setup

Owner: Adarsh · **Support:** all

- Repo with `frontend/`, `backend/`, `data/`, `docs/`; `.gitignore` (no `.env`, `node_modules`, `__pycache__`); `README`; branch rules; `PROGRESS.md`.
- Everyone runs the empty app locally. **Accept:** all six can clone and run both apps; a first commit is on `main`. **Verify:** each member posts a screenshot of the running app.

Phase 1: Research and assumptions

Owner: Rohitha · **Support:** Adarsh

- Study AICTE's structure, RB and PGRC roles, and complaint types from public AICTE sources.

- Document assumptions to confirm: official statuses, SLAs, identifiers, roles, notification channel.
- A one-page problem summary and the glossary. **Accept:** `docs/assumptions.md` lists every assumption with a source or a "to verify" mark. **Verify:** Adarsh checks each claim has a link or is marked as an assumption.

Phase 2: Data model and API contract

Owner: Adarsh · **Support:** Rohitha

- Final schema from section 5; Pydantic models; an OpenAPI list of endpoints with example JSON.
- Mock JSON files so the frontend can start immediately. **Accept:** every screen has at least one endpoint and a sample response. **Verify:** frontend members build one page against the mock JSON.

Phase 3: PPT submission (parallel track)

Owner: Priya · **Support:** Satya Kamal, Rohitha (references), Adarsh (review)

- Six slides in the SIH format: title, idea and solution, technical approach, feasibility and viability, impact and benefits, research and references.
- Tone: evidence first, measured claims only, real links.
- Include one architecture diagram and one workflow diagram. **Accept:** every reference opens; every number is measured or labelled as a target. **Verify:** Adarsh and one other member do a "judge" review and list weaknesses.

Phase 4: Synthetic data

Owner: Shreeshanth · **Support:** Adarsh (scenario spec), Rohitha (reviews 10 random rows)

- Adarsh writes a short scenario spec first: which overlaps and near-misses to plant.
- Generator script: about 40 institutions, about 300 cases across RB, PGRC and other cells, with noisy descriptions and reference numbers. Realistic wording may be AI-drafted but must be read and fixed by hand, with no real people or institutions' real complaints.
- Planted scenarios: clear overlaps, near-misses (same institution, unrelated topic), missing IDs, wrong dates.
- A labelled pair file (at least 200 pairs, related or not, including hard cases) for the matching evaluation. Whoever writes the matching rules does not build this file, so the evaluation is not circular.
- CSV import of the seed data. **Accept:** a seeded database; a `data/README` explaining that all data is synthetic. **Verify:** query counts by Bureau and status; spot-check 10 random cases.

Phase 5: Backend core

Owner: Rohitha · **Support:** Adarsh

- Institution and case CRUD, institution profile endpoint with consolidated history.
- Pydantic validation on every input.
- Auto-generated case numbers `{BUREAU}-{YYYY}-{00001}` (for example `RB-2026-00219`) from a per-Bureau, per-year counter updated inside a transaction. Unique constraint; never reused; immutable after a transfer. Imported cases keep their own numbers. **Accept:** Swagger shows all endpoints; invalid input gives a clean error; creating 100 cases in a loop gives 100 unique, consecutive numbers. **Verify:** pytest for create, read, update; fetch one institution and see all its cases.

Phase 6: Workflow and action history

Owner: Sohana

- Append-only `case_events`; endpoints for status change, transfer, escalation, remark.
- Updating a case writes an event and updates `last_action` and `last_action_date`. **Accept:** history is never overwritten. **Verify:** transfer one case RB → PGRC and see the event with a timestamp and both Bureaus.

Phase 7: Search and filters

Owner: Shreeshanth · **Support:** Sohana

- Search by institution name, Permanent ID, case ID. Filters: Bureau, region, category, status, date range; combinable and clearable.
- Pagination and sensible empty responses. **Accept:** combined filters return correct results. **Verify:** 10 test queries with expected counts written down beforehand.

Phase 8: Design system and wireframes

Owner: Satya Kamal · **Support:** Priya

- Colours, typography, status chips, tables, cards, layout; wireframes for all six screens in the playbook (overview, institution directory and profile, case list and detail, overlap queue, reports, admin).
- Accessible contrast; a responsive desktop layout. **Accept:** wireframes approved by Adarsh. **Verify:** a team walkthrough of the clickable wireframe.

Phase 9: Frontend shell and overview dashboard

Owner: Satya Kamal · **Support:** Priya

- App layout, navigation, role switcher (mock), KPI cards (total, pending, under process, disposed, transferred, overdue), Bureau workload chart, recent alerts. **Accept:** every number comes from the API. **Verify:** KPI totals equal the case-list totals for the same filters.

Phase 10: Case list/detail and institution profile

Owner: Priya · **Support:** Satya Kamal (design review)

- Case list with filters, case detail (owner, status, last and next action, timeline), institution profile with all related cases. **Accept:** from an institution profile you can reach a case and see its history. **Verify:** trace one case from profile to history on screen; record it.

Phase 11: Overlap detection, stage 1 (rules)

Owner: Adarsh

- Candidate pairs by Permanent ID across Bureaus; reasons recorded; dedupe so one pair is flagged once.
- Missing or conflicting IDs go to a manual-verification list. **Accept:** planted overlaps are flagged; near-misses show Low or are not flagged. **Verify:** run on synthetic data; list the flagged pairs and check against the planted truth.

Phase 12: Overlap detection, stage 2 (similarity, explainability, evaluation)

Owner: Adarsh · **Support:** Sohana

- Add category match, shared references, text similarity (TF-IDF + rapidfuzz; optional embeddings), date proximity.
- Confidence band from transparent rules; reasons list per pair.
- Evaluation script on the labelled pairs: precision, recall, and the baseline comparison. Tune rules on a development split only and report on a held-out test split. **Accept:** a results table in `docs/eval.md` with the real numbers. **Verify:** a second person reruns the script and gets the same numbers.

Phase 13: Overlap review queue UI

Owner: Satya Kamal · **Support:** Adarsh

- Queue of pairs with reasons, both cases side by side, decision buttons (related, duplicate, not related, needs review), coordination note, link or dismiss.
- Decisions write to `overlap_links` and to both cases' histories. **Accept:** no merge and no auto-close; both case IDs stay. **Verify:** review one pair end to end and see it in both timelines.

Phase 14: Alerts and SLA

Owner: Sohana

- Alert types: overlap, pending reminder, escalation, SLA warning. Configurable rule (for example due within 3 days), labelled "demo rule, not an official SLA".
- Alert list with priority and status; dismiss and acknowledge. **Accept:** changing the config changes which alerts appear. **Verify:** set a case due in 2 days; confirm the alert; change the config; confirm it disappears.

Phase 15: Reports and analytics

Owner: Sohana · **Support:** Shreeshanth (export)

- Bureau-wise, region-wise, institution-wise and status-wise reports; pending-case ageing buckets; disposal rate; overlap review outcomes; monthly summary; CSV export. **Accept:** report totals equal the underlying records. **Verify:** recompute three totals with a direct SQL query and compare.

Phase 16: Roles, audit log and data quality

Owner: Sohana · **Support:** Shreeshanth

- Roles: Bureau Officer, Coordinator, Senior Management (read-only), Administrator. Role-based visibility in API and UI.
- Audit log of sensitive actions.
- Data-quality checks: missing IDs, conflicting institution details, impossible dates, unowned cases. **Accept:** each role sees only its permitted screens and actions. **Verify:** log in as each role and try one forbidden action; capture the denial.

Phase 17: Integration, regression and hardening

Owner: Adarsh · **Support:** all

- Run the playbook's success checklist, the demo flow three times start to finish, and fix issues.
- Fix known problems: hard-coded backend URLs (use env vars), error states, empty states. **Accept:** the checklist is fully ticked with evidence. **Verify:** each member runs the flow on a clean clone and reports.

Phase 18: Demo, video and deployment

Owner: Priya · **Support:** Satya Kamal (video editing), Adarsh

- 3-minute script: institution profile → dashboard → overlap queue → link without merging → coordination note and history → updated report.
- A recorded backup video; a deployed link; a rehearsal with a timer, at least three runs.
- Judge Q&A sheet, each answer owned by one member. **Accept:** the live demo and the video both finish in 3 minutes. **Verify:** a rehearsal log with times and what broke.

8. Definition of done (every phase)

- [ ] Meets the phase's acceptance checks, with evidence attached
- [ ] Reviewed by Adarsh and merged to `main`
- [ ] No hard-coded secrets or URLs
- [ ] `PROGRESS.md` updated with verified vs. unverified items
- [ ] Nothing in the UI shows placeholder or raw unresolved values

9. Risks

Risk	Mitigation
Matching looks like "just keyword search"	Show reasons plus the measured precision/recall against the baseline
Real AICTE rules unknown	Label every rule as an assumption; list questions for the problem owner
Backend skills uneven	Hard modules (11, 12) with Adarsh; easier ones with pairing
Late integration	Mock JSON on day 1; weekly full-flow run
Demo breaks	Backup video; seeded data; rehearsal
Overclaiming on slides	Only measured numbers, everything else labelled "target"

10. Judge questions to prepare

- How is this different from a normal ticketing dashboard?
- How do you decide two cases overlap, and what if you are wrong? (Human review; reasons shown; no auto-merge.)
- What if the institution ID is missing? (Manual verification list.)
- How accurate is the matching? (Real precision/recall on labelled synthetic pairs; say it is synthetic.)
- How would this connect to AICTE's existing systems? (API or approved export behind one adapter; nothing claimed.)
- How do you handle privacy and roles? (Role-based access, audit log, synthetic data only.)